Reinforcement Learning

Assignment #01

SAPIENZA
UNIVERSITÀ DI ROMA

Info

- Deadline: October 26th, 2022

- Students may discuss assignments, but the solutions must be typed and coded up **individually**

- Students must **indicate the names of colleagues they collaborated with**

Folder Organization

- The assignment source code will be available on Classroom
- You will find:
  - assignment1.pdf: with all the information
  - assignment1.zip that contains:
    - requirements.txt
    - ilqr/
      - main.py (do not touch!)
      - env.py (do not touch!)
      - **student.py**
    - prolicy_iteration/
      - main.py (do not touch!)
      - grid_world.py (do not touch!)
      - **student.py**

Theory Submission

The theory solutions must be submitted in a pdf file named “XXXXXXX.pdf”, where XXXXXXX is your matricula.

We encourage you to **type equations on an editor**, rather than uploading scanned files

Use the pdf file also to **comunicate the students you collaborated with** and to **insert a small report of the code exercises**

Code Submission

The code solutions must be submitted in a zip file named “XXXXXXX.zip”, where XXXXXXX is your matricula.

The zip file must be organized exactly as the original assignment.zip file. Wrongly submitted assignments will be penalized.

Only edit the “students.py” files

Theory

1. Demonstrate the convergence of the Policy Iteration Algorithm.
2. Given the following environment settings
   - States: $\{s_i : \ i \in \{1...7\}\}$
   - Reward:

$$r(s,a) = \begin{cases} +1 & \text{if } s = s_1 \\ +10 & \text{if } s = s_7 \\ 0 & \text{otherwise} \end{cases}$$

   - Dynamics: $p(s_6|s_6, a_1) = 0.5$, $p(s_7|s_6, a_1) = 0.5$
   - Policy: $\pi(s) = a_1 \ \ \forall s \in S$

   and the value function at the iteration $k = 1$:

$$V_k = [1,\ 0,\ 0,\ 0,\ 0,\ 0,\ 10],$$

   with $\gamma = 0.5$.

   Compute $V_{k+1}(s_6)$

Code: Policy Iteration

Implement the policy iteration algorithm on the grid world environment

```python
def policy_iteration(env, gamma=0.99, iters=100):
    policy = np.zeros(env.num_states, dtype=np.int)

    # policy = ...

    return policy.reshape(env.height, env.width)
```

```python
def main():
    parser = argparse.ArgumentParser(description='Run tests.')
    parser.add_argument('--render', action='store_true')
    args = parser.parse_args()
    render = args.render

    rewards = []
    for i in range(3):
        print(f"Starting game {i+1}")
        env = NonDeterministicGridWorld(4+i,5+i)
        policy = policy_iteration(env)

        state = env.reset()
        if render: env.render()

        total_reward = 0.
        done = False
        while not done:
            action = policy[state[0],state[1]]
            state, reward, done, _ = env.step(action)
            total_reward += reward
            if render: env.render()
        print("\tTotal Reward:", total_reward)
        rewards.append(total_reward)

    print("Mean Reward: ", np.mean(rewards))
```

SAPIENZA
UNIVERSITÀ DI ROMA

Code: Policy Iteration

Implement the iLQR on the cartpole environment

$$\begin{cases} x_{t+1} = x_t + \dot{x}_t \delta_t \\ \dot{x}_{t+1} = \dot{x}_t + \ddot{x}_t \delta_t \\ \theta_{t+1} = \theta_t + \dot{\theta}_t \delta_t \\ \dot{\theta}_{t+1} = \dot{\theta}_t + \ddot{\theta}_t \delta_t \end{cases}$$

$$\ddot{x}_t = \frac{u_t + L_p \dot{\theta}_t^{\,2} sin(\theta_t) - L_p \ddot{\theta}_t cos(\theta_t)}{M}$$

$$\ddot{\theta}_t = \frac{g sin(\theta_t) - cos(\theta_t) \frac{u_t + L_p \dot{\theta}_t^{\,2} sin(\theta_t)}{M}}{L(\frac{4}{3} - \frac{M_p cos^2(\theta_t)}{M})}$$

Code: Policy Iteration

Implement the iLQR on the cartpole environment

```python
class CartPole:
    def __init__(self, env, x=None,max_linear_velocity=2,
        max_angular_velocity=np.pi/3):
        if x is None:
            x = np.zeros(2)
        self.env = env

    def getA(self, u, x=None):
        if x is None:
            x = self.x
        x, x_dot, theta, theta_dot = x
        polemass_length = self.env.polemass_length
        gravity = self.env.gravity
        masspole = self.env.masspole
        total_mass = self.env.total_mass
        length = self.env.length
        force = u
        dt = self.env.tau
        A = np.eye(4)

        # A = ...

        return A

    def getB(self, x=None):
        if x is None:
            x = self.x

        x, x_dot, theta, theta_dot = x
        polemass_length = self.env.polemass_length
        gravity = self.env.gravity
        masspole = self.env.masspole
        total_mass = self.env.total_mass
        length = self.env.length
        dt = self.env.tau

        B = np.zeros((4,1))

        # B = ...

        return B
```

```python
def lqr(A, B, T=100):
    K = np.zeros((1,4))

    # K = ...

    return K
```

```python
def main():
    parser = argparse.ArgumentParser(description='Run tests.')
    parser.add_argument('--render', action='store_true')
    args = parser.parse_args()
    render = args.render

    rewards = []
    if render:
        env = CartPoleEnv(render_mode='human')
    else:
        env = CartPoleEnv()
    for i in range(3):
        print(f"Starting game {i+1}")
        x,_ = env.reset()

        total_reward = 0
        xf = np.array([0., 0., 0., 0.])

        model = CartPole(env, x=x)
        u = np.zeros(1)
        done = False
        while not done:
            error = x - xf
            B = model.getB(x=x)
            A = model.getA(u, x=x)
            K = lqr(A, B)
            u = K@error
            x, reward, done, _, _ = env.step(u)
            total_reward += reward
        print("\tTotal Reward:", total_reward)
        rewards.append(total_reward)
```

SAPIENZA
UNIVERSITÀ DI ROMA